CSA0622

Sayyad Ahmad Alisha
(192525415)

CO1 AT2

Q36) Consider the concept map : Algorithm → operation count - Growth function → Big-O → upper Bound map how counting operations leads to asymptotic upper bounds - Extend the map with worst case Analysis Justify why Big-O represents worst-case behavior Interpret how this supports algorithm comparison.

Ans Explanation

Algorithm → operation count

An algorithm performs a sequence of operations we count the number of basic operations to measure its efficiency

② Operation count → Growth function

The operation count expressed as a function of input size n such as

n, $n \log n$, n^2

This is called the growth function

③ growth function → Big-O

Big-O notation represents the asymptotic growth of the algorithm

It ignores constant and lower order terms focusing on how the algorithm grows for large inputs

④ Big-O → upper bound.

Big-O gives the upper bound on running time

It show the maximum time an algorithm may take as the input size increases.

Why Big O represents worst-case Behaviour:

Big-O guarantees that the running time will not exceed the specified upper bound.

It helps estimate the maximum execution time making it useful for designing reliable and efficient algorithm

How this supports Algorithm comparision:

Big-O allows different algorithms to be compared independently of hardware or programing language An algorithm with smaller Big-O complexity is generally more efficient for large inputs

Example

Linear search : O(n)

Binary search : O(log n)

Since O(log n) grows much slower than O(n) Binary search is more efficient for large datasets

Conclusion

Computing operation leads to growth functions which is expressed using Big-O notation. Big-O provides the asymptotic upper bound and represents the worst case behaviour of an algorithm This make it easy to compare algorithms and chose most effective one for large input sizes

⑤ expansion → substitution method

The substitution method repeatedly expands the recurrence and substitutes smaller terms until a general formula is obtained

This method is commonly used to solve recurrence relations.

Importance of Pattern recognition

Pattern recognisation helps to identify how the recourence grows

It makes solving recurrences faster and more accurate.

How this supports solving recurrences

Expanding the recurrence reveals pattern

The pattern is converted into a summation

The summation is solved to determine the final time complexity

Thus recurrence expansion and the substitution method provide a systamatic way to analyze recurive algorithms

Conclusion

Recurrence expansion reveals repeating patterns which are simplified using the substitution method and summation. Recognizing these patterns is essential for determining the algorithms time complexity and comparing the efficiency of recursive algorithms

Qus) consider the concept map : Recurrence Relation Expansion → Patteren → summation → complexity explain how recurrence expansion reveals patterns extend the map with substitution method link justify the importance of pattern recognisation Interpet how this supports solving recurrences

Ans Recurrence Relation → Expansion → Pattern → summation → complexity → substitution method.

Explanation

① Recurrence relation → expansion

A recurrence relation expresses the running time of an algorithm in terms of smaller input sizes It expanded by repeatedly replacing recursive terms until the base case is reached

② Expansion → Pattern :

During expansion a repeating sequence of terms appear.

Identifying the pattern make it easier to write the recurrence in general form.

③ Pattern → summation

The repeated pattern is expressed as a summation Summation combines all repeated terms into a single mathematical expression

④ Summation complexity

solving the summation gives the final asymptotic time complexity such as O(n), O(nlogn) & O(n²)